

复现 Grounding DINO：开放词汇目标检测 与视觉定位

苏思齐 (12310645)，杨官宇涵 (12313614)，莫丰源 (12311805)
南方科技大学 (SUSTech)，深圳

https://github.com/YangGuanyuhan/CV_Project_2026_S

2026 年 6 月 21 日

摘要

开放词汇目标检测将传统检测扩展到任意文本描述类别，使模型能够检测超出固定训练词汇表的物体。本报告复现了 Grounding DINO——一个最先进的开放词汇检测器，并在 COCO 2017 验证集上进行零样本评估。使用官方 Swin-Tiny 骨干网络权重，我们在完整的 val2017（5,000 张图像）上取得了 $AP=0.4055$ ($AP_{50}=0.5317$) 的结果。我们进一步进行了检测阈值敏感度和提示词格式设计的消融实验，发现点分隔的提示词格式对性能至关重要，且将 `box_threshold` 从 0.35 降低到 0.25 可将子集 AP 从 0.4382 提升到 0.4637。成功和失败案例的定性分析表明，小目标检测和拥挤场景混淆是主要的局限性。

1 项目概述

1.1 什么是开放词汇目标检测？

传统目标检测系统（如 YOLO、Faster R-CNN）只能检测训练时定义好的固定类别（例如 COCO 的 80 类）。如果要检测新类别，就需要收集标注数据并重新训练模型——这是一个昂贵且耗时的工作。

开放词汇目标检测解决了这个问题：给定任意文本描述（如”金色的狗. 花盆. 木椅子.”），模型就能检测出对应的物体，不需要针对这些类别进行额外训练。

1.2 Grounding DINO 简介

Grounding DINO 是 2024 年 ECCV 发表的一个开放词汇检测模型，由 IDEA Research 开发。它结合了两个强大的技术：

- **DINO**：一个基于 Transformer 的端到端目标检测器，用”目标查询”(object queries) 替代传统的手工锚框
- **BERT**：一个预训练的语言模型，能将文本编码为上下文相关的向量表示

通过可变形注意力机制将视觉特征和文本特征融合，Grounding DINO 可以检测任意文本描述的物体。

1.3 本项目做了什么

我们使用官方发布的 `groundingdino-py` Python 包和 Swin-Tiny 预训练权重，完成以下工作：

1. 在 COCO 2017 验证集（5,000 张图像，80 个类别）上进行完整的零样本评估
2. 通过消融实验分析检测阈值和提示词格式对性能的影响
3. 对检测成功和失败案例进行定性分析
4. 构建模块化的 Python 推理、评估和可视化流水线

我们没有做：从头训练、在 COCO 上微调、修改模型架构。

2 模型架构

2.1 三大核心组件

1. Swin Transformer 视觉骨干网络

- 将输入图像处理为 4 个不同分辨率的层次化特征图
- 使用移位窗口注意力机制，计算复杂度与图像大小呈线性关系
- 配置：窗口大小 = 7，嵌入维度 = 96，各阶段深度 = [2, 2, 6, 2]

2. BERT 语言编码器

- 使用 bert-base-uncased 版本（12 层 Transformer，隐藏维度 768）
- 将文本提示词编码为上下文相关的词向量
- 最大文本长度：256 个 token

3. 可变形注意力融合模块

- 6 层跨模态融合，每层 8 个注意力头
- 让 900 个目标查询同时关注视觉特征位置和文本 token 位置
- 每个查询输出：边界框、置信度分数、关联的文本短语

2.2 推理流程

1. **加载模型**：加载 662MB 的预训练权重文件和架构配置
2. **图像预处理**：BGR→RGB 转换，缩放到短边 800px（长边不超过 1333px），ImageNet 归一化
3. **文本编码**：BERT 分词器将提示词转为 token，编码为 $(L, 768)$ 的文本特征矩阵
4. **跨模态检测**：视觉特征与文本特征通过可变形注意力联合处理，输出检测结果
5. **后处理**：归一化坐标 $(cx, cy, w, h) \rightarrow$ 像素坐标 $(x_1, y_1, x_2, y_2) \rightarrow$ COCO 格式 (x, y, w, h)

3 实验设置

3.1 数据集

使用 **COCO 2017 验证集**:

- 5,000 张 RGB 图像（室内、室外、街道等真实场景）
- 80 个常见物体类别（人、自行车、汽车、狗……）
- 36,781 个标注边界框（平均每张图约 7.4 个物体）
- 类别分布呈长尾形态：常见类如”人”有大量实例，稀有类如”吹风机”不到 100 个

3.2 实验环境

表 1: 实验环境

项目	配置
GPU	NVIDIA RTX 4060 Laptop, 8GB VRAM
CPU	13 代 Intel Core i7, 16 核
内存	32 GB
Python	3.10.20
PyTorch	2.7.1 + CUDA 11.8
groundingdino-py	0.4.0

3.3 评估指标

使用 COCO 标准指标（pycocotools COCOeval）:

- **AP**: 10 个 IoU 阈值 (0.50–0.95) 上的平均精度——主要指标
- **AP50**: IoU=0.50 时的 AP（宽松定位）
- **AP75**: IoU=0.75 时的 AP（严格定位）

- **APS / APM / APL:** 按物体大小（小/中/大）分解的 AP
- **AR@k:** 每张图取 k 个检测时的平均召回率

4 实验结果

4.1 主要结果：完整 COCO val2017

表 2: 完整 COCO val2017 评估结果（5,000 张图像）

精度指标	数值	召回指标	数值
AP	0.4055	AR@1	0.3273
AP50	0.5317	AR@10	0.4756
AP75	0.4422	AR@100	0.4875
APS (小)	0.2587	ARS (小)	0.3156
APM (中)	0.4328	ARM (中)	0.5154
APL (大)	0.5510	ARL (大)	0.6472

关键发现:

- 模型在 4,925/5,000 张图像中检测到物体（覆盖率 98.5%），共产生 39,133 个检测
- 75 张图像（1.5%）没有任何检测结果——通常是物体太小或遮挡严重的情况
- AR@100 (0.4875) > AP (0.4055): 模型能识别很多物体但定位不够精确，无法通过更严格的 IoU 阈值
- 大物体性能 (APL=0.551) 是小物体 (APS=0.259) 的约 **2.1** 倍

4.2 消融实验 A：阈值敏感度

测试了三种阈值设置（500 张图像子集）:

表 3: 阈值敏感度实验结果

box_th	text_th	AP	AP50	APS	APL
0.25	0.20	0.4637	0.5884	0.3061	0.6338
0.35	0.25	0.4382	0.5532	0.2650	0.6269
0.45	0.30	0.3931	0.4827	0.2092	0.5841

分析:

- 阈值越低，AP 越高，但检测数量也大幅增加
- 小物体对阈值最敏感：APS 从 0.209 提升到 0.306（+46%）
- 大物体对阈值不敏感：APL 仅从 0.584 提升到 0.634（+8.5%）
- 最佳阈值 box_th=0.25 的 AP 比默认值高约 2.5 个百分点

4.3 消融实验 B：提示词格式

测试了三种提示词格式（500 张图像子集）:

表 4: 提示词格式对比

格式	示例	AP	AP50
点分隔	person . bicycle . car .	0.4382	0.5532
逗号分隔	person, bicycle, car,	0.1671	0.2038
句子风格	There are person, bicycle...	0.1675	0.2051

核心发现: 点分隔格式比其他格式高出约 **62% 的 AP**。原因在于 BERT 的分词器——点号周围有空格时被识别为独立的分隔 token，清晰划分了类别边界。逗号格式中逗号可能与前面的词合并（如“car,” 变成一个 token），句子格式中的虚词则引入了噪声。

4.4 定性分析

4.4.1 成功案例

模型能准确检测中大型物体，框与物体贴合紧密，类别标签正确。典型成功场景包括：户外街道（人、汽车、交通灯）、室内空间（椅子、桌子、杯子）、运动场景（人物、球拍、球）。

4.4.2 失败模式

1. **小物体漏检（最常见）**：小于 32×32 像素的物体经常被漏掉。根本原因是 Swin-T 骨干网络的 $32 \times$ 下采样导致小物体在特征图中几乎没有表示。
2. **拥挤场景混淆（偶发）**：在物体密集的场景中（ >15 个实例），模型可能产生重复框或类别混淆。
3. **遮挡导致的定位误差（中等频率）**：部分遮挡降低检测置信度，并降低边界框精度。这解释了 AP50(0.53) 与 AP75(0.44) 之间的 9 个百分点的差距。

5 讨论

5.1 与官方结果的对比

官方论文报告的 Swin-T 零样本 AP 约为 0.485，而我们得到的是 0.4055。这个约 8 个百分点的差距主要来自四个因素：

1. 官方使用 GLIP 风格的评估协议（仅测试未见过的类别），我们使用标准 COCO 协议（全部 80 类）
2. 官方可能不使用固定阈值，而我们使用了 `box_th=0.35` 进行过滤
3. 官方可能使用更丰富的提示词（类别描述、同义词），我们只用了标准类别名称

4. `groundingdino-py` 包版本可能与原始代码有细微差异

我们的结果 ($AP=0.4055$) 代表了使用固定阈值的标准 COCO 评估条件下更真实的零样本基线。

5.2 主要局限性

- 小物体检测: $APS=0.259$ vs $APL=0.551$, 这是最关键的瓶颈
- 提示词格式敏感: 点分隔格式是强制要求, 自然语言查询效果很差
- 计算成本: 完整评估需要约 68 分钟 (0.81 秒/张), 不适合实时应用
- 零检测图像: 1.5% 的图像完全无检测输出

5.3 未来方向

1. 在 COCO 训练集上微调 (预期可将 AP 提升到约 0.506)
2. 跨数据集评估 (Objects365、OpenImages、LVIS)
3. 与其他开放词汇检测器 (GLIP、OWL-ViT、YOLO-World) 的公平对比
4. 自适应阈值策略以恢复零检测图像
5. 模型蒸馏或 TensorRT 部署以降低推理延迟

6 项目实施

6.1 代码架构

本项目实现为一个配置驱动的 Python 包 (`cv-project-2026`), 包含约 1,800 行源代码和 10 个 CLI 脚本:

表 5: 项目模块组织

模块	职责	代码行数
src/models/grounding_dino.py	模型包装器	182
src/inference/predictor.py	推理接口	229
src/inference/visualizer.py	可视化绘制	262
src/inference/coco_visualizer.py	COCO 对比可视化	213
src/engine/evaluator.py	COCO 评估引擎	368
src/datasets/coco_categories.py	类别映射与提示词	269
src/datasets/coco_dataset.py	数据集工具	188
src/utils/box_ops.py	框操作 (IoU, NMS)	115

6.2 运行命令速查

环境安装

```
conda create -n grounding_dino python=3.10 -y
conda activate grounding_dino
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
pip install groundingdino-py
pip install -r requirements.txt
```

单张图像推理

```
python scripts/inference.py --image demo.jpg \
    --text "person . car . dog ." \
    --checkpoint checkpoints/groundingdino_swint_ogc.pth
```

完整 COCO 评估

```
python scripts/eval.py --config configs/grounding_dino.yaml \
    --checkpoint checkpoints/groundingdino_swint_ogc.pth \
    --coco_image_dir data/coco/val2017 \
    --coco_ann_file data/coco/annotations/instances_val2017.json
```

消融实验

```
python scripts/run_experiments.py \  
    --checkpoint checkpoints/groundingdino_swint_ogc.pth \  
    --coco_image_dir data/coco/val2017 \  
    --coco_ann_file data/coco/annotations/instances_val2017.json \  
    --subset_size 500
```

7 结论

本项目成功复现了 Grounding DINO 的零样本开放词汇目标检测能力。
核心成果总结：

1. **基线性能**：在 COCO 2017 val2017 (5,000 张图像) 上达到 AP=0.4055, AP50=0.5317
2. **阈值影响**：AP 在 0.39–0.46 之间随阈值变化，最佳 box_th=0.25
3. **提示词格式至关重要**：非点分隔格式导致约 62% 的 AP 下降
4. **小物体是主要瓶颈**：APS=0.259 远低于 APL=0.551 (2.1 倍差距)
5. **定位优于分类**：AR@100=0.4875 > AP=0.4055，说明类别识别尚可但定位精度不足

成员贡献

苏思齐 (12310645)：文献调研、提示词消融实验设计与执行、失败案例定性分析。

杨官宇涵 (12313614)：整体项目设计与实现，包括全部 Python 模块和 CLI 脚本编写、完整 COCO 评估执行、消融实验执行、报告撰写。

莫丰源 (12311805)：阈值消融实验设计与执行、实验数据分析、可视化图表生成、报告实验部分撰写。